

Bases teóricas de estructuras de almacenamiento en memoria. GA3-220501093-
AA3-EV01.

Instructor: Libardo Gómez Díaz

Aprendiz: Juan Camilo Herrera Castellanos

Ficha: 3336085

SENA

Servicio Aprendizaje Nacional

2026



Índice

1. Introducción
2. Lenguajes Compilados vs. Interpretados
3. Características de JavaScript
4. Tipos de Datos Primitivos
5. Operadores en JavaScript
6. Referencias Bibliográficas

1. Introducción

Los fundamentos de programación son esenciales para comprender cómo se desarrollan las aplicaciones informáticas y cómo los computadores procesan la información. El conocimiento de los lenguajes de programación permite crear soluciones que automatizan tareas, gestionan datos y facilitan la interacción entre usuarios y sistemas tecnológicos.

Entre los diferentes lenguajes de programación, JavaScript se ha convertido en una de las herramientas más utilizadas para el desarrollo web debido a su versatilidad y capacidad de ejecutarse tanto en el navegador como en servidores. Para aprovechar sus funcionalidades de manera eficiente, es importante comprender cómo administra los recursos del sistema, especialmente la memoria.

La gestión de memoria en JavaScript se realiza de forma automática mediante mecanismos como la asignación dinámica de memoria y la recolección de basura (Garbage Collector), lo que permite liberar espacios que ya no están siendo utilizados por el programa. Conocer estos procesos ayuda a desarrollar aplicaciones más eficientes, optimizar el rendimiento y evitar problemas relacionados con el consumo excesivo de memoria. Por esta razón, el estudio de las estructuras de almacenamiento en memoria constituye una base fundamental para cualquier programador que desee comprender el funcionamiento interno de los programas y crear soluciones de software de mayor calidad.

2. Diferencias entre lenguajes compilados e interpretados

Aspecto	Lenguajes Compilados	Lenguajes Interpretados
Traducción del código	El código se traduce completamente antes de ejecutarse.	El código se traduce y ejecuta al mismo tiempo.
Herramienta utilizada	Compilador	Intérprete
Archivo ejecutable	Genera un archivo ejecutable independiente.	No genera un ejecutable independiente.
Velocidad de ejecución	Generalmente más rápida.	Generalmente más lenta.
Detección de errores	Los errores se detectan durante la compilación.	Los errores se detectan durante la ejecución.
Uso de recursos	Suele optimizar mejor los recursos del sistema.	Puede consumir más recursos al interpretar el código constantemente.
Portabilidad	Puede requerir recompilación para otro sistema operativo.	Suele ser más fácil de ejecutar en diferentes plataformas con un intérprete compatible.
Ejemplos	C, C++, Go, Rust.	JavaScript, Python, PHP, Ruby.

3. Características principales de JavaScript

JavaScript es uno de los lenguajes de programación más importantes de la actualidad y constituye una tecnología fundamental para el desarrollo de la web moderna. Gracias a su flexibilidad y amplio uso, permite crear páginas dinámicas, aplicaciones interactivas y servicios web que funcionan tanto en navegadores como en servidores.

Característica	Descripción
Interpretado	JavaScript es ejecutado por un motor de JavaScript integrado en los navegadores web o mediante entornos como Node.js, sin necesidad de compilar el código manualmente antes de su ejecución.
Basado en prototipos	Utiliza un sistema de herencia basado en prototipos, diferente al modelo tradicional de orientación a objetos basado en clases, aunque actualmente también permite el uso de clases para facilitar el desarrollo.
Multiparadigma	Permite aplicar diferentes estilos de programación, incluyendo programación funcional, imperativa y orientada a objetos, brindando gran flexibilidad al desarrollador.
Dirigido por eventos (Event-Driven)	Está diseñado para responder a eventos como clics, movimientos del mouse, pulsaciones de teclas o solicitudes de red, lo que lo hace ideal para crear interfaces interactivas.
Débilmente tipado	No es necesario declarar explícitamente el tipo de dato de una variable, ya que JavaScript lo determina automáticamente durante la ejecución del programa.

Estas características han convertido a JavaScript en uno de los lenguajes más utilizados en el desarrollo de aplicaciones web modernas, permitiendo la creación de soluciones dinámicas, eficientes y adaptables a diferentes entornos tecnológicos.

4. Tipos de datos primitivos en JavaScript

Los tipos de datos primitivos son los valores más básicos que maneja JavaScript. Se caracterizan por ser inmutables, lo que significa que su valor no puede modificarse una vez creado; cualquier cambio genera un nuevo valor. Además, estos datos suelen almacenarse directamente en la memoria de tipo **Stack (pila)**, lo que permite un acceso rápido y eficiente.

Tipo de dato	Descripción	Ejemplo
String	Representa cadenas de texto formadas por caracteres.	"Hola Mundo"
Number	Representa valores numéricos, incluyendo números enteros y decimales.	10, 3.14
Boolean	Representa valores lógicos que pueden ser verdadero o falso.	true, false
Undefined	Indica que una variable ha sido declarada pero aún no tiene un valor asignado.	let edad;
Null	Representa la ausencia intencional de un valor o un objeto vacío.	let usuario = null;
Symbol	Tipo de dato introducido en ES6 que permite crear identificadores únicos.	Symbol("id")
BigInt	Permite trabajar con números enteros extremadamente grandes, superando el límite del tipo Number.	12345678901234567890n

Los tipos de datos primitivos son fundamentales en JavaScript porque constituyen la base para almacenar y manipular información dentro de los programas. Su naturaleza inmutable y su almacenamiento eficiente en memoria contribuyen al buen rendimiento de las aplicaciones desarrolladas con este lenguaje.

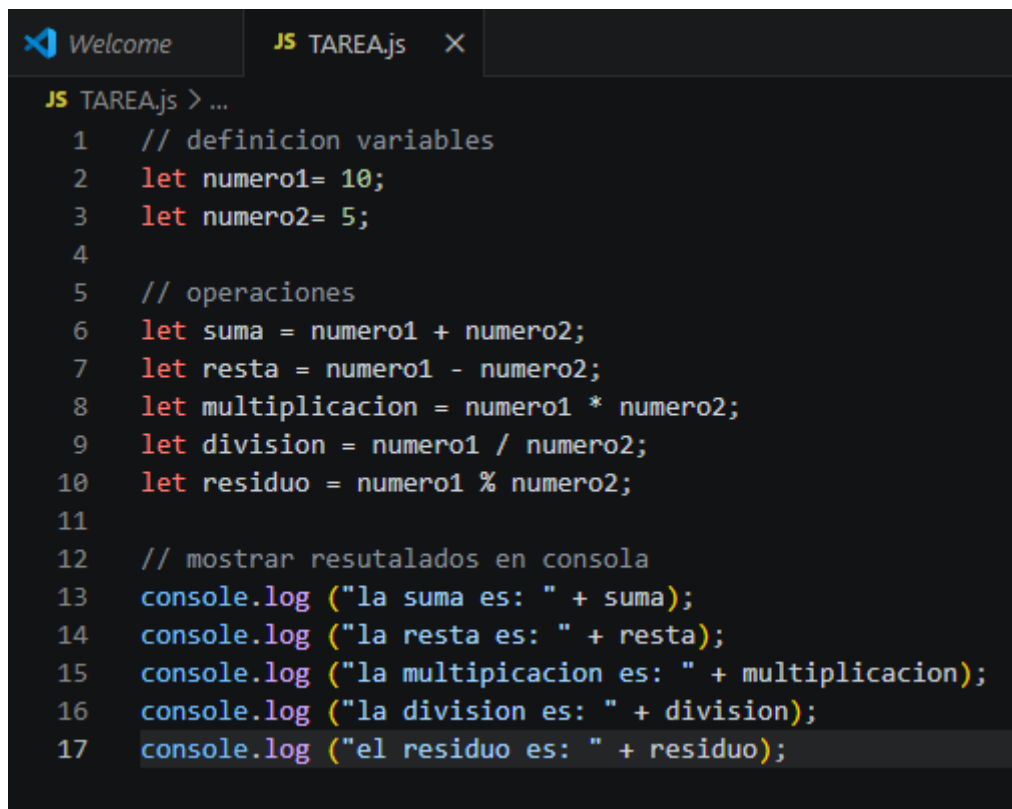
5. Operadores en JavaScript

Los operadores son símbolos especiales que permiten realizar operaciones sobre variables y valores dentro de un programa. En JavaScript, los operadores se clasifican en diferentes categorías según la función que desempeñan.

Tipo de operador	Operadores	Descripción	Ejemplo
Aritméticos	+, -, *, /, %, ++, --	Permiten realizar operaciones matemáticas.	10 + 5 = 15
Asignación	=, +=, -=, *=, /=	Se utilizan para asignar o actualizar valores en una variable.	x += 5
Comparación	==, ===, !=, >, <, >=, <=	Comparan valores y devuelven un resultado lógico (true o false).	10 > 5 → true
Lógicos	&&, `	Se utilizan para combinar, evaluar o negar condiciones dentro de una expresión.	, !`

Descripción de los operadores principales

Aritméticos: Se utilizan para realizar cálculos matemáticos como suma, resta, multiplicación, división, módulo (residuo de una división) e incrementos o decrementos de una unidad.



```

JS TAREA.js > ...
1  // definicion variables
2  let numero1= 10;
3  let numero2= 5;
4
5  // operaciones
6  let suma = numero1 + numero2;
7  let resta = numero1 - numero2;
8  let multiplicacion = numero1 * numero2;
9  let division = numero1 / numero2;
10 let residuo = numero1 % numero2;
11
12 // mostrar resultados en consola
13 console.log ("la suma es: " + suma);
14 console.log ("la resta es: " + resta);
15 console.log ("la multiplicacion es: " + multiplicacion);
16 console.log ("la division es: " + division);
17 console.log ("el residuo es: " + residuo);

```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Advertencia: PowerShell detectó que es posible que estés usando un lector de pantalla y que hayas deshabilitado PSReadLine con el módulo PSReadLine".

PS C:\Users\camil\Documents\ADSO> node TAREA.js
la suma es: 15
la resta es: 5
la multiplicacion es: 50
la division es: 2
el residuo es: 0
PS C:\Users\camil\Documents\ADSO> |
```

Asignación: Permiten almacenar valores en variables y modificar su contenido de forma abreviada mediante operadores compuestos.

```
JS tarea2.js > ...
1  let contador = 10;
2  contador +=5;
3  contador *= 2;
4
5  console.log ("igual a " + contador)|
```

```
Advertencia: PowerShell detectó que es posible que estés usando un lector de pantalla y que hayas deshabilitado PSReadLine".

PS C:\Users\camil\Documents\ADSO> node tarea2.js
igual a 30
PS C:\Users\camil\Documents\ADSO> |
```

Comparación: Se emplean para verificar relaciones entre valores. Es importante destacar que == compara únicamente el valor, mientras que === compara tanto el valor como el tipo de dato.

```
JS TAREA.js JS tarea2.js JS tarea3.js X
JS tarea3.js > ...
1  let variable1 = 10;
2  let variable2 = 5;
3
4  console.log (variable1 === variable2); // false 10 no es igual a 5
5  console.log (variable1 !== variable2); // true 10 es diferente de 5
6  console.log (variable1 > variable2); // true 10 es mayor que 5
7  console.log (variable1 < variable2); // false 10 no es menor que 5
8  console.log (variable1 >= variable2); //true 10 es mayor o igual a 5
9  console.log (variable1 <= variable2); // false 10 no es menor o igual a 5
10
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Advertencia: PowerShell detectó que es posible que est
odule PSReadLine".

PS C:\Users\camil\Documents\ADSO> node tarea3.js
false
true
true
false
true
false
PS C:\Users\camil\Documents\ADSO>
```

Lógicos: Son utilizados para evaluar múltiples condiciones. El operador && requiere que todas las condiciones sean verdaderas, || requiere que al menos una lo sea y ! invierte el resultado lógico de una expresión.


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Advertencia: PowerShell detectó que es posible que el módulo PSReadLine".

PS C:\Users\camil\Documents\ADSO> node tarea4.js
Puede conducir? true
acceso total? true
inverso de tiene licencia: false
PS C:\Users\camil\Documents\ADSO> 
```

6. Referencias Bibliográficas

Mozilla Developer Network (MDN Web Docs). (s.f.). *JavaScript: fundamentos de programación*. <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>

Servicio Nacional de Aprendizaje (SENA). (2026). *Componente formativo: Bases teóricas de estructuras de almacenamiento en memoria GA3-220501093-AA3-EV01*.

Autor desconocido. (2023). *Bases teóricas de estructuras de almacenamiento en memoria GA3-220501093-AA3-EV01*. Scribd. <https://es.scribd.com/document/652287595/BASES-TEORICAS-DE-ESTRUCTURAS-DE-ALMACENAMIENTO-EN-MEMORIA-GA3-220501093-AA3-EV01>

GeeksforGeeks. (s.f.). *Difference between compiler and interpreter*. <https://www.geeksforgeeks.org/difference-between-compiler-and-interpreter/>

GeeksforGeeks. (s.f.). *JavaScript tutorial*.

<https://www.geeksforgeeks.org/javascript-tutorial/>